Conservative Adaptive Bit-Width Narrowing for LLVM Memory Objects

AdaptiveWidthMemoryPass Artifact Draft - Generated PDF

Abstract. Integer values stored in memory often occupy fewer significant bits than their declared source-level type. While scalar integer narrowing is a familiar compiler optimization theme, narrowing memory objects is more delicate because the transformation must preserve load/store semantics, pointer use structure, and signedness restoration. This paper presents a conservative LLVM New Pass Manager prototype, *AdaptiveWidthMemoryPass*, that narrows selected local stack arrays and internal/private globals when all observed stores can be classified by local bit-width inference. The pass rewrites memory object element types, truncates stores, and restores narrowed loads with either zero-extension or sign-extension according to a tracked extension family. Unsupported pointer uses, mixed signed/unsigned value families, and unknown stored values are rejected rather than speculated on. On the current fixture corpus, the prototype identifies 28 memory-object candidates, transforms 20 of them, and records a static summed per-element width reduction of 234 bits. On 32 C-derived kernels, it transforms 26 memory-object candidates and exposes the expected tradeoff between narrower memory and additional cast instructions.

1. Introduction

Many programs store values in integer containers wider than the value range required at a particular program point. Examples include masked counters, small enumerations, byte-derived values stored in 16- or 32-bit arrays, and sign-extended offsets later stored in wider integer memory. Narrowing such memory objects can reduce static storage width and may enable downstream optimizations. However, memory narrowing is more constrained than scalar narrowing: every load and store must be rewritten consistently, pointer uses may escape, and signedness-sensitive values must be restored with the correct extension operation.

This work develops a small, inspectable LLVM pass that prioritizes correctness and reproducibility over broad coverage. The prototype narrows only integer arrays allocated on the stack and internal/private integer globals. It accepts only direct pointer uses through getelementptr, load, and store. It performs local stored-value inference over constants, casts, masks, simple select/phi/or compositions, and LLVM KnownBits information. If a candidate cannot be proven safe and profitable under these rules, it is left unchanged and reported with a reason.

- a conservative memory-object narrowing transformation for LLVM IR;
- an explicit extension-family safety policy for zero- and sign-restored loads;
- deterministic candidate classification and rejection logging;
- a reproducible metrics harness that reports coverage, transformation counts, width deltas, and IR overhead.

2. Motivation

A local array whose stored values are masked before every store may not require its declared element width. The pass can replace a 16-bit array with a five-bit array, truncate stores, and zero-extend loads back to the original type.

```
arr = alloca [2 x i16]
masked = and i16 x, 31
store i16 masked, ptr slot
v = load i16, ptr slot

; after narrowing
arr.narrow = alloca [2 x i5]
masked.narrow = trunc i16 masked to i5
store i5 masked.narrow, ptr slot.narrow
v.narrow = load i5, ptr slot.narrow
v.wide = zext i5 v.narrow to i16
```

For signed byte-derived values, zero-extension is not equivalent. Therefore the prototype tracks whether the stored-value family must be restored by zext or sext. A candidate containing a mixture of both families is rejected.

3. Safety Model

Let a candidate memory object have original element type iN and proposed narrow element type iK where K < N. The transformation is allowed only if the memory object is a supported stack array or internal/private global, all pointer users are recursively limited to supported GEP/load/store operations, every store has an inferred width at most K, every stored value agrees on one extension family, and the candidate is profitable.

The local preservation condition is: extend_E(trunc_K(v)) = v for each stored value v that may be loaded from the object. The current implementation checks this condition through syntactic and KnownBits-based inference rules rather than whole-program analysis. If inference fails, the candidate is treated as unknown and is not transformed.

Invariant and Preservation Sketch

For every transformed object O, every value stored through a supported pointer path to O belongs to one inferred extension family E and has representable width at most K, the new element width. Because all admitted loads and stores are rewritten through the same rebuilt pointer graph, each transformed load observes a value previously stored in truncated form. Since inference requires extend_E(trunc_K(v)) = v for every admitted stored value, restoring the narrowed load with E preserves the original loaded value. Escaping pointers, unsupported casts, aggregate loads, volatile/atomic behavior, unknown values, and mixed extension families are rejected before rewriting.

4. Algorithm and Implementation

The pass is implemented as an LLVM 18 New Pass Manager plugin. It exposes full analysis/transformation, log-only analysis, stack-allocation-only transformation, and global-only transformation pipelines. Candidate discovery scans functions for local stack integer arrays and the module for internal/private integer globals. Scalar allocas, floating-point objects, external globals, aggregate loads, and pointer escapes are rejected.

The inference engine computes a required bit width and extension family for stored values. It recognizes integer constants, truncation including C-style trunc(and i32 %x, mask) forms, zero-extension and sign-extension chains, constant masks, bitwise OR, select and bounded phi nodes when all arms agree, and LLVM KnownBits as a conservative fallback. For compositions such as select, phi, and OR, operands must agree on the same extension family.

For a profitable candidate, the pass creates a replacement memory object with the narrowed integer element type. It rebuilds GEPs using the new source element type, rewrites stores by truncating stored values to the new type, rewrites loads from the new type, and extends loaded values back to the original type.

Transformation Algorithm

```
for object O in stack_arrays + internal_globals:
  if not integer_memory_object(O):
    reject(O, "non-integer")
    continue
  U = collect_supported_pointer_uses(O)
  if U has escaping or unsupported users:
    reject(O, "unsupported-user")
    continue
  W, E = infer_width_and_extension(stores(O))
  if inference failed:
    reject(O, "unknown-width")
    continue
  if W >= original_element_width(O):
    reject(O, "not-profitable")
    continue
  O2 = create_narrow_object(O, W)
  rewrite_geps_loads_stores(O, O2, E)
  erase_dead_old_ir(O)
```

Pass Architecture

The implementation separates candidate discovery, recursive pointer-use classification, width and extension-family inference, and IR rewriting. The metrics harness consumes the same deterministic log output used by the tests, so paper tables are generated from the artifact rather than manually transcribed.

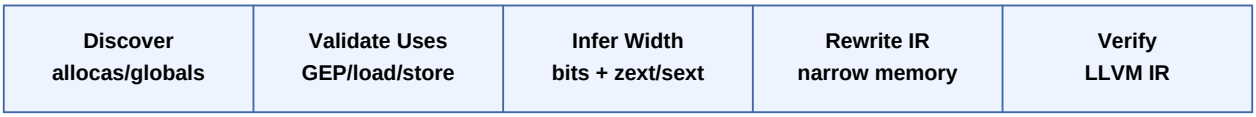

Figure 1. Pass architecture. Unsupported, unknown, or unprofitable candidates are logged and left unchanged.

5. Evaluation Methodology

The artifact includes three evaluation layers. Fixture-level tests exercise individual transformation and rejection behaviors. A benchmark-metrics harness compiles small C kernels to LLVM IR with Clang 18, applies a light mem2reg normalization step to remove irrelevant scalar spills, runs the pass, verifies transformed IR, and emits tab-separated and Markdown results. Runtime differential harnesses compile baseline and transformed benchmark executables and compare deterministic checksum outputs. Metrics include candidate counts, safety classifications, transformed globals and allocas, rewritten loads and stores, static summed per-element bit savings, LLVM IR instruction-count delta before and after LLVM default O2, and coarse process-level pass wall time.

Input	Cand.	Safe	Blocked	Unknown	Trans. G	Trans. A	Loads	Bits
candidates	10	8	2	2	2	3	2	67
transform_alloca	7	7	0	2	0	5	2	50
inference	6	6	0	0	0	6	5	59
stack_positive	1	1	0	0	0	1	1	11
globals.example	2	2	0	0	2	0	2	36
masked_stack.example	1	1	0	0	0	1	1	11
rejected_escape.example	1	0	1	0	0	0	0	0
Total	**28**	**25**	**3**	**4**	**4**	**16**	**13**	**234**

Table 1. Fixture-level evaluation summary. Bit savings are static per-element width deltas, not measured runtime footprint reductions.

Benchmark	Cand.	Safe	Blocked	Trans.	RW	Raw d	O2 d	Bits
alias_pointer_reject	1	0	1	0	0	+0	+0	0
alpha_blend_weights	1	1	0	1	7	+7	+0	8
checksum_nibbles	1	1	0	1	8	+8	+0	12
embedded_counters	1	1	0	1	8	+8	+0	11
fir4_coefficients	1	1	0	1	8	+4	+0	13
global_constants	3	2	1	2	2	+2	+0	25
global_flag_table	3	3	0	3	3	+3	+0	42
global_lookup_small	1	1	0	0	0	+0	+0	0
grayscale_threshold	1	1	0	1	8	+8	+0	8
ipv4_header_fields	1	1	0	1	8	+8	+0	8
mixed_signed_reject	1	1	0	0	0	+0	+0	0
moving_average4	1	1	0	1	8	+8	+0	6
overflow_uncertainty	1	1	0	0	0	+0	+0	0
packet_flags	1	1	0	1	8	+8	+0	11
pixel_channels	1	1	0	1	8	+8	+0	8
protocol_version_table	1	1	0	1	8	+8	+0	11
quantized_filter	1	1	0	1	6	+6	+0	9
rejection_cases	3	2	1	0	0	+0	+0	0
rgb565_unpack	1	1	0	1	6	+6	+0	10
saturating_sensor	1	1	0	1	4	+4	+0	7
select_ranges	1	1	0	1	4	+4	+4	10
sensor_bucket	1	1	0	1	4	+4	+0	6
signed_delta_pair	1	1	0	1	4	+4	+0	8
signed_offsets	1	1	0	1	4	+4	+0	8
signed_select_offsets	1	1	0	0	0	+0	+0	0
signed_state_delta	1	1	0	0	0	+0	+0	0
small_state_machine	1	1	0	1	6	+6	+0	13
status_flags	1	1	0	1	6	+6	+0	13
tcp_flag_decode	1	1	0	1	12	+12	+0	15
unsigned_histogram	2	2	0	1	5	+5	+0	9
volatile_reject	1	1	0	0	0	+0	+0	0
wide_unknown_reject	1	1	0	0	0	+0	+0	0
Total	**39**	**36**	**3**	**26**	**145**	**+141**	**+4**	**271**

Table 2. C-kernel benchmark metrics. Raw delta is measured immediately after rewriting; O2 delta is measured after LLVM default O2 on baseline and transformed IR.

6. Results

Across seven fixture inputs, the pass identifies 28 candidates. It classifies 25 as safe under the pointer-use policy and transforms 20 total memory objects: 4 globals and 16 stack arrays. The fixture corpus intentionally includes unknown-width and escaped-pointer cases; these account for the 3 blocked candidates and 4 unknown-width classifications. The resulting static summed per-element width delta is 234 bits.

The C-kernel benchmark snapshot demonstrates the intended behaviors at source-derived IR level. The suite includes unsigned mask extraction, pixel-channel packing, packet flag decoding, signed offsets, global constants, select-based ranges, and deliberate rejection cases. Across 32 kernels, the pass sees 39 memory-object candidates, classifies 36 as safe under the use policy, and transforms 26 objects. The transformed C kernels increase raw static IR instruction count by 141 assignments because the pass inserts explicit truncation and extension instructions; after LLVM default O2 this overhead falls to 4 assignments.

Runtime differential testing compiles baseline and transformed versions of all 32 benchmark harnesses and compares deterministic checksum outputs. The current run reports 32 passing comparisons and 0 failures. This does not replace formal proof or large input-space testing, but it strengthens the artifact-level behavioral evidence for the current benchmark suite.

7. Limitations

The current prototype does not handle arbitrary pointer aliasing, external globals, scalar allocas, bitcast/address-space-cast rewrite cases, aggregate loads, volatile or atomic semantics, interprocedural effects, or broad arithmetic range inference. The current evaluation uses small fixtures and kernels rather than external benchmark suites. Therefore the reported numbers establish correctness and reproducibility of the artifact, not general application-level memory or performance improvements.

8. Related Work

The implementation builds on LLVM's modular pass infrastructure and uses LLVM IR semantics as the transformation target. Its inference style is related to classic data-flow and abstract-interpretation ideas, and to compiler propagation analyses such as conditional constant propagation. The use of LLVM KnownBits connects the prototype to existing value-tracking infrastructure. Unlike scalar-only simplifications, this work focuses specifically on narrowing memory object element types while preserving load/store behavior.

9. Future Work

Future work should scale the metrics harness to larger benchmark suites, measure optimized machine-code effects after downstream LLVM passes, support additional safe pointer-use patterns, extend arithmetic range inference, and add runtime differential testing for transformed kernels. Another important direction is comparing memory-object narrowing against scalar bit-width optimizations to determine when memory rewriting provides unique benefits.

10. Conclusion

This draft presented a conservative LLVM prototype for adaptive bit-width narrowing of selected memory objects. The pass combines local stored-value inference, explicit signedness restoration, conservative pointer-use filtering, and deterministic reporting. The current results show that the implementation can safely transform targeted fixture and C-derived kernels while preserving a clear rejection policy. The artifact is now positioned for broader benchmark evaluation and iterative paper development.

References

1. Chris Lattner and Vikram Adve. LLVM: A Compilation Framework for Lifelong Program Analysis and Transformation. CGO, 2004.
2. Patrick Cousot and Radhia Cousot. Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints. POPL, 1977.
3. Mark N. Wegman and F. Kenneth Zadeck. Constant Propagation with Conditional Branches. ACM TOPLAS, 1991.
4. LLVM Project. LLVM Language Reference Manual and ValueTracking/KnownBits infrastructure documentation.